

ExplainSomatic — The Complete Technical Reference

This document exists so that anyone — you, a teammate, a panel member — can read it and understand exactly what every file does, why every number was chosen, and what the data actually looks like at every stage. Nothing here should require re-reading the code to verify; where something is a hypothesis rather than a proven fact, that's said explicitly.

PART 1 — What This Project Actually Does, In One Paragraph

Given a patient's tumour-only DNA sequencing data (no matched blood sample), ExplainSomatic looks at every genomic position with any hint of a mutation, represents the surrounding evidence two different ways (a local pileup "image" and a wider sequence-context window), fuses those two views with a neural network, and decides whether each position is a real cancer-causing mutation or just noise — paying deliberate extra attention to mutations present in only a small fraction of reads, since those are exactly the kind that signal an emerging drug-resistant tumour subclone before it's otherwise detectable.

PART 2 — The Core Data Objects, Explained and Visualized

2.1 What a BAM file is, concretely

A BAM file is a list of sequencing reads, each one saying "this exact 150-letter DNA fragment aligned to chromosome 21, starting at position 15,043,201." Every read is independent evidence about what the DNA actually says at that stretch of the genome.

2.2 What the Truth VCF is, and why we trust it

`high-confidence_sSNV_in_HC_regions_v1.2.1.vcf.gz` is SEQC2's answer key — a list of genomic positions where a large international effort (many sequencing centers, many replicates, cross-validated) concluded "yes, this position really does carry a somatic mutation in HCC1395." We use it for exactly one purpose: **deciding the binary label** (variant / not variant) for every candidate site. We do NOT use it for the numeric VAF value anymore — see Part 2.4, this was an actual bug we fixed.

A VCF line looks roughly like this (simplified):

#CHROM	POS	REF	ALT
chr21	15043201	G	A
chr21	15049385	C	T

Meaning: "at chromosome 21, position 15,043,201, the reference genome says G, but in this tumour, it's really A."

2.3 What VAF is, restated precisely

VAF (Variant Allele Frequency) = (reads showing the mutant base) ÷ (total reads covering that position). It is a property of **the actual sequencing data**, not something you look up — which is exactly why we now compute it directly from reads rather than trusting an annotation (Part 2.4).

2.4 Low-VAF vs. High-VAF — where the 5% line comes from

We define **low-VAF** as **VAF < 5%**. This threshold isn't arbitrary or invented for this project — it comes from the clinical motivation in the survey: resistance mutations like EGFR T790M are documented to emerge and sit in the 1–5% range before becoming clinically obvious on a scan. 5% is the boundary in the literature between "still hiding" and "starting to dominate." Everything below that line gets 5× the loss weight during training (`losses.py`, Part 6).

2.5 The VAF bug, restated precisely (for completeness)

Originally, `real_data.py` tried to read a `VAF` or `AF` field from the truth VCF's INFO column. The real file doesn't carry one under either name. Every real variant silently got a hardcoded fallback of `vaf=0.5`. We fixed this by computing VAF directly from the actual reads at each site instead — which is also, arguably, the more correct approach regardless, since VAF is fundamentally an empirical property of the sequencing data, not a database annotation.

PART 3 — Candidate Generation (`candidate_generation.py`), Fully Explained

3.1 Why this step exists at all

A genome has ~3.2 billion positions (chr21 alone: 46,709,983). Running a CNN+Transformer on every single one is computationally impossible at any reasonable budget. So the first step is a **cheap, high-recall prefilter**: scan the whole chromosome fast, using simple counting (not a neural network), and only keep positions worth the expensive model's attention.

3.2 Line-by-line walkthrough

```
python

for pileup_col in bam.pileup(region=region, min_base_quality=10, ...):
```

This asks `pysam` to hand us, one genomic position at a time, every read that overlaps it (`min_base_quality=10` throws out very low-confidence individual base calls before we even count them).

```
python
```

```

depth = 0
alt_count = 0
for pileup_read in pileup_col.pileups:
    if pileup_read.is_del or pileup_read.is_refskip: continue
    aln = pileup_read.alignment
    if aln.is_duplicate or aln.is_secondary or aln.is_qcfail or aln.is_unmapped:
        ...
    depth += 1
    read_base = aln.query_sequence[qpos].upper()
    if read_base != ref_base and read_base in "ACGT":
        alt_count += 1

```

For every read touching this position: skip it if it's a duplicate read (PCR artifact), a secondary/supplementary alignment (the read maps ambiguously elsewhere too), or failed the sequencer's own quality check. Otherwise, count it toward depth, and separately count it if its base disagrees with the reference.

```

python

if depth < min_depth: continue
alt_frac = alt_count / depth if depth else 0.0
if alt_count >= min_alt_reads and alt_frac >= min_alt_frac:
    ...
    candidates.append(...)

```

The actual filter. A position becomes a candidate only if **both** conditions hold — enough raw reads disagree (not just one, which is almost always sequencing error) **and** that's a large enough fraction of the local depth.

3.3 Why these exact threshold numbers ((2 alt reads), (1% alt fraction), (depth ≥ 8))

This is a heuristic, not a value taken from a specific published paper — be honest about that if asked. The reasoning behind each number:

- **(min_alt_reads=2)**: a single mismatching read is extremely common sequencing noise (base-call error rate is roughly 0.1-1% per base even on good instruments) — requiring at least 2 independent reads to agree sharply cuts single-read-error false candidates, at very low cost to recall (a real variant, unless it's at essentially zero depth, will almost always show up in more than one read).
- **(min_alt_frac=0.01) (1%)**: set **deliberately at/below** the 5% low-VAF threshold from Part 2.4 — if we set this higher than 5%, we'd be filtering out the exact low-VAF mutations

Objective 2 cares about before the model ever saw them. 1% gives some margin below the 5% target.

- `min_depth=8`: below 8 reads of coverage, the statistics are too thin to say anything meaningful — a "1 out of 4 reads mismatch" (25%) position with only 4x depth is far less trustworthy than the same fraction at 60x depth. 8 is a conservative floor, not a rigorously derived number.

What we haven't done yet, and should: actually measure what fraction of the truth VCF's real variants survive this filter (a "candidate recall" check). If real low-VAF variants are being dropped here, no amount of model improvement downstream can recover them — this is flagged as unresolved in `DATA_PREPROCESSING_QA.md` and remains unresolved as of this document.

3.4 The high-confidence regions filter (added later)

```
python

if regions_by_chrom is not None and not in_regions(chrom, pos, regions_by_chrom, s
    n_dropped_outside_regions += 1
    continue
```

Uses `bisect` (binary search) against a sorted list of `(start, end)` genomic intervals to cheaply check "is this position inside a region SEQ2 actually validated." On our real chr21 run, this dropped 109,958 of the 301,264 raw candidates (~36%) — meaning over a third of raw mismatch signal sat in regions where "not in the truth VCF" wouldn't have meant "confirmed negative," just "untrusted territory."

PART 4 — Building the Tensors (`real_data.py`), Fully Explained and Visualized

4.1 The pileup tensor — what it actually looks like

Shape: `(5, 64, 21)` — 5 channels, 64 reads, 21 genome positions. Here is a REAL example, generated from the actual code (Channel 0, showing only 12 of the 64 reads for readability), for a genuine 3% VAF variant:

```

Channel 0 (1=matches reference, 0=mismatch, -1=no coverage here)
read 0:  [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
read 1:  [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
read 2:  [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
read 3:  [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
read 4:  [1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]    <- mismatch, but NOT the
candidate column (col 10)
read 5:  [-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1]    <-
read doesn't cover this window at all
read 6:  [1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1]    <- mismatch AT the
candidate column (col 10) = real signal
read 7:  [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
read 8:  [-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1]
read 9:  [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
...

```

The candidate column (index 10, dead center of the 21-wide window) is column 10, i.e. `column[10]` above. Notice: only read 6 actually shows a mismatch there — that's the "2-3 reads out of 64" signal that defines a low-VAF call, and it's genuinely subtle sitting next to reads 4's unrelated off-center mismatch (which is just ordinary noise elsewhere in the window, irrelevant to the call).

Channel 1 (base quality, normalized 0-1), first 5 reads, real values:

```

read 0: [0.85 0.93 0.77 0.78 0.92 0.85 0.81 0.86 0.84 0.87 0.88 0.75 0.99 ...]
read 1: [0.89 0.79 0.86 0.87 0.78 0.80 0.79 0.84 1.00 0.94 0.73 0.76 0.76 ...]

```

These are the sequencer's own per-base confidence scores, rescaled to 0-1 by dividing the raw Phred score by 40 (the typical practical maximum for Illumina).

Channels 2, 3, 4 are, respectively: strand (0 or 1, per read, constant across that read's whole row), mapping quality (0-1, also constant per read), and a simple marker that's 1.0 only in the center column (so the network always knows exactly which column is "the" candidate position, rather than having to infer it).

4.2 Why 64 reads specifically

A CNN needs a **fixed-size** input tensor — you can't feed it "sometimes 40 rows, sometimes 200 rows." Real sequencing depth varies (SEQC2's HCC1395 at high aggregate depth can have far more than 64 reads at a position); we pad with `-1` sentinel rows when there are fewer, and simply **truncate** when there are more. 64 was chosen as a reasonable middle value for typical WGS depth (~30-60x is common) without being so large it wastes tensor space on mostly-

padding at low-depth sites. **Real cost of this choice:** at very high-depth positions, reads beyond the first 64 are simply thrown away — possible information loss, not yet measured.

4.3 Why a 21-position pileup window

Be honest about this one: it's our own scaled-down choice, not a literature-matched number. DeepVariant's actual published pileup image width is 221 positions (100bp flanking each side). We used 21 (10bp each side) specifically to keep tensor size and training compute small during prototyping — $5 \times 64 \times 21 = 6,720$ values per example at 21-wide, versus $5 \times 64 \times 221 = 70,720$ at DeepVariant's real width — roughly **10.5× more data per example** at the literature width. This is a real, acknowledged scale-down, flagged as worth revisiting once the pipeline itself is proven correct.

4.4 The sequence-context tensor — what it looks like

Shape: $(301,)$ — a flat sequence of integers, each 0-3, representing 301 consecutive reference bases (± 150 bp around the candidate, this width IS literature-matched, from TransSSVs). Real example, first 30 of 301 tokens: $[0\ 3\ 2\ 1\ 1\ 3\ 0\ 2\ 0\ 0\ 2\ 3\ 2\ 3\ 2\ 3\ 2\ 0\ 3\ 1\ 2\ 1\ 0\ 3\ 3\ 2\ 1\ 3\ 2\ 1]$ — decoding via A=0,C=1,G=2,T=3, this is the literal DNA sequence $\text{ATGCCTAGAAGCTGTGTGAGTCGATTGCTGC...}$ read directly off the reference genome at that location. Note this is the **reference** sequence, not per-read — Stream B looks at genomic *context*, not individual read evidence (that's Stream A's job).

4.5 Why ± 150 bp specifically

Unlike the 21-wide pileup window, this number **is** literature-anchored — it matches the context window TransSSVs uses (cited directly in the survey), chosen because that's an already-validated window size for catching structurally complex variants via sequence context alone.

4.6 What "good enough compute" would actually look like (scaling up from the prototype)

Setting	Current (prototype)	Literature / full spec	What changing it costs
Pileup window width	21	221 (DeepVariant's real default)	~10.5× more pileup tensor data per example
Reads per pileup	64	Depth-adaptive or higher fixed cap (128+)	More GPU memory per batch, proportionally
Transformer layers	6 (already full spec)	6	— already matches Objective 1's target
Chromosomes	1 (chr21)	Whole genome (22 autosomes + X/Y)	Roughly 46.7M / ~3.1B ≈ 66× more genome to scan; realistically needs a multi-GPU or extended multi-session compute budget, not a free-tier session
Dataset (BAM replicates)	1 of 63 available	Multiple replicates for robustness	Each replicate adds another full candidate-gen + caching pass

Bottom line on compute: the model itself is small (~170K params) — it's not GPU-memory-hungry. The real cost of scaling up is **data volume** (more chromosomes = more candidate generation + caching time, which is I/O-bound, not GPU-bound) and **wider tensors** (if the 21→221 pileup width change is made, that's a straightforward but real increase in per-example memory and compute).

PART 5 — The Model Itself (`models.py`), Fully Explained

5.1 Why only 2 convolutional layers in Stream A

```
python

self.conv = nn.Sequential(
    nn.Conv2d(in_channels, 32, kernel_size=3, padding=1),
    nn.BatchNorm2d(32), nn.ReLU(inplace=True),
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64), nn.ReLU(inplace=True),
)
```

Two reasons, honestly stated: (1) our pileup tensor is already small (64×21) — a deep image-classification-style CNN (10+ layers, built for 224×224 natural images) would be overkill and would shrink the spatial dimensions to nothing after a few pooling/stride operations; (2) this is the prototyping phase, and a shallow CNN is faster to train and easier to debug (which mattered a lot this session, given how many real bugs we found and needed to isolate). **This is a scoped-down choice, not a claim that 2 layers is architecturally optimal** — worth experimenting with 3-4 layers once the pipeline's correctness is fully trusted.

5.2 How the CNN actually processes one example, step by step

1. Input: $(5, 64, 21)$ — a mini "image" with 5 color channels instead of the usual 3 (RGB).
2. First `Conv2d`: slides a 3×3 window across the whole 64×21 grid, learning 32 different simple pattern detectors (e.g., "a mismatch surrounded by matches" might become one learned pattern).
3. `BatchNorm` + `ReLU`: stabilizes training, adds non-linearity.
4. Second `Conv2d`: combines those 32 simple patterns into 64 more complex ones, still at the same 64×21 spatial size (`padding=1` keeps dimensions unchanged).
5. `AdaptiveAvgPool2d(1)`: collapses the entire 64×21 grid down to a single number per channel (64 numbers total) by averaging — this is what turns "a whole pileup image" into "one summary vector."
6. `Linear(64, 128)`: projects that summary up to a 128-dimensional vector — Stream A's final output.

5.3 How the Transformer (Stream B) actually processes the data — from scratch

What self-attention means, concretely, with our numbers: we have 301 positions, each turned into a 64-dimensional vector (`Embedding(4, 64)`). Self-attention computes, for every one of those 301 positions, a weighted average of *all 301 positions' vectors* — where the weights are learned based on how "relevant" each other position is to this one. Concretely, each position produces three vectors: a **Query** ("what am I looking for"), a **Key** ("what do I offer"), and a **Value** ("what information do I actually carry"). The attention weight between position i and position j is computed as $\text{softmax}(\text{Query_i} \cdot \text{Key_j})$ across all j — high when Query_i and Key_j point in a similar direction, low otherwise. Position i 's new representation becomes the weighted sum of all Value_j , weighted by those scores.

Why "8 heads": instead of computing one single Query/Key/Value relationship, the model computes **8 separate, smaller ones in parallel** (each head is $64/8 = 8$ -dimensional), letting different heads specialize — one head might learn to track "am I in a repetitive region," another might track "how far am I from the candidate site," etc. Their outputs get concatenated back to 64 dimensions.

Why "6 layers": each layer applies one round of this attention process, then a small feedforward network, then passes the result to the next layer. Stacking 6 of these lets the model build up increasingly abstract representations — layer 1 might catch simple local patterns, layer 6 might represent something like "this whole region looks like a homopolymer run." This number **is** the literature-matched value (matches TransSSVs), unlike the CNN's layer count.

Positional encoding: since self-attention has no inherent sense of order (it treats the 301 positions as an unordered set by default), we add a $\sin/\cos$ -based positional encoding vector to each position's embedding before the attention layers, so the model can tell position 5 apart from position 250.

What comes out: `tokens` — all 301 positions' final representations — and `center_token`, specifically the representation at position 150 (dead center, the candidate site itself), pulled out for use downstream.

5.4 Cross-attention fusion, explained with our exact tensor shapes

```
python
```

```
q = self.query_proj(a_vec).unsqueeze(1) # (batch, 1, 64)
fused, attn_weights = self.attn(q, b_tokens, b_tokens) # b_tokens: (batch, 301)
```

Stream A's 128-dim vector gets projected down to 64 dimensions and treated as a single Query. Stream B's full 301-position output becomes both the Keys and Values. The attention mechanism computes, for this one query, **how relevant each of the 301 context positions is to the pileup evidence**, and returns a weighted combination — this is literally "the pileup evidence asking the sequence context which parts of it matter right now." The `attn_weights` (shape `(batch, 1, 301)`) are saved and are exactly the kind of thing that could be visualized alongside Grad-CAM for a fuller explainability picture (not currently wired into a report, but the data's there).

5.5 Final combination and classification head

```
python
```

```
combined = torch.cat([a_vec, fused_ctx, b_center], dim=1) # 128 + 64 + 64 = 256
logit = self.head(combined).squeeze(1)
```

All three pieces of evidence — the raw CNN summary, the fused context-aware summary, and the transformer's own center-position embedding — get concatenated into one 256-dimensional vector and passed through a small feedforward network (`(256 → 64 → 1)`) ending

in a single number (the logit): positive means "leaning toward variant," negative means "leaning toward not."

PART 6 — Why Only sSNV, Restated With the Code Reason

`real_data.py`'s `extract_pileup_and_context()` compares single bases:

```
python

match = 1.0 if base == local_ref else 0.0
```

There is no code path that looks at a read's CIGAR string to detect an insertion or deletion — the entire tensor representation assumes "one base, one column." An indel truth-VCF entry would get compared against this single-base-only representation and either be silently mishandled or require a structurally different encoding. Using only the sSNV truth file keeps what we're training on consistent with what the model can actually represent. Extending to indels is real future work requiring both a new CIGAR-aware tensor design and the separate indel truth VCF SEQC2 also publishes.

PART 7 — Grad-CAM (`gradcam.py`), Line by Line

```
python

logit = model(pileup, ctx_tokens)
feat_map = model.stream_a.last_feature_map # (1, 64, 64, 21) -- stashed during t
feat_map.retain_grad()
score = logit.sum()
model.zero_grad()
score.backward()
```

Runs one example through the model normally, but keeps a copy of Stream A's last convolutional feature map (before pooling) and tells PyTorch to remember its gradient (normally only leaf tensors keep gradients; this forces an exception for this one intermediate tensor). Then it computes gradients of the output score with respect to that feature map — this tells us "if this particular pixel in the feature map had been bigger, how much would the final score have changed."

```
python
```

```
weights = grads.mean(dim=(2, 3), keepdim=True) # per-channel importance
cam = F.relu((weights * feat_map).sum(dim=1)) # (1, reads, width)
```

Averages each of the 64 channels' gradients down to one importance score per channel, uses those as weights to combine the 64 channels' feature maps into a single (64, 21) heatmap, and zeroes out any negative values (ReLU) — we only care about evidence that pushed *toward* "variant," not away from it. The result, after normalizing to 0-1, is exactly the kind of heatmap that could overlay the original pileup: brighter cells = reads/ positions that mattered most to this specific call.

PART 8 — The Training Script (train_production.py), Explained

8.1 Resumability

Every epoch, full state (model weights, optimizer state, scheduler state, epoch number, best-metric-so-far) is saved to last.pt. On restart with --resume, this is loaded back and the loop continues from epoch+1 — this is what made it safe to survive multiple Colab disconnects this session without losing progress.

8.2 Mixed precision (torch.autocast + GradScaler)

Runs most of the forward pass in 16-bit floating point instead of 32-bit — faster on a real GPU, at a small, generally-negligible precision cost. The GradScaler compensates for a known numerical issue where very small gradients can round to zero in 16-bit ("underflow") by temporarily scaling the loss up before the backward pass, then scaling back down before the optimizer step.

8.3 Balanced sampling (added this session — the batch-starvation fix)

```
python

weight_pos = args.target_positive_fraction / max(n_pos, 1)
weight_neg = (1.0 - args.target_positive_fraction) / max(n_neg, 1)
sampler = WeightedRandomSampler(sample_weights, num_samples=len(train_ds), replace
```

Every training example gets a sampling weight — positives get a weight that, in aggregate, makes them show up roughly target_positive_fraction (currently 15%) of the time, even though their true rate is 0.227%. This is sampling **with replacement**, meaning the same rare positive example can legitimately be drawn multiple times within one epoch — necessary given there are only 304 of them.

8.4 Threshold calibration (added this session)

After training, the best checkpoint's predictions on validation data are swept across many possible decision thresholds, and whichever threshold maximizes F1 is kept and saved — replacing the arbitrary default of 0.5, which was never validated as the right cutoff for this level of class imbalance.

8.5 Model selection logic

Saves `best.pt` using low-VAF recall as the primary metric **only when validation actually has low-VAF positives to measure** (`n_low_vaf > 0`); falls back to overall F1 otherwise — necessary because on chr21, val has zero low-VAF positives, and without this fallback no `best.pt` would ever be produced at all.

PART 9 — The Repair Script (`repair_vaf.py`), Explained

Recomputes VAF for already-cached examples **without touching the BAM again** — it reads the pileup tensor's channel 0 (match/mismatch), center column, across all 64 reads, counts how many are covered (`!= -1`) and how many of those are mismatches, and divides. This is mathematically identical to what `extract_pileup_and_context()` computes live from real reads, just reconstructed from data already sitting in the cache — verified correct against 4 hand-checked cases (including one with partial read coverage) before being used on real data.

PART 10 — What Every Colab Command Actually Did

Command	What it does
<code>drive.mount('/content/drive')</code>	Connects Colab's filesystem to your Google Drive, so files written to <code>/content/drive/MyDrive/...</code> persist across sessions (unlike <code>/content</code> itself).
<code>!apt-get install samtools sra-toolkit bwa tabix bcftools</code>	Installs the standard bioinformatics command-line tools this pipeline depends on — <code>samtools</code> (BAM manipulation), <code>bwa</code> (alignment, unused since we had a pre-aligned BAM), <code>tabix</code> / <code>bcftools</code> (VCF indexing/querying).
<code>!pip install pysam h5py</code>	Python bindings for BAM/VCF access (<code>pysam</code>) and our tensor cache format (<code>h5py</code>).
<code>!curl -L -o ... 'https://api.gdc.cancer.gov/data/...'</code>	Downloads the reference genome — a direct, verified GDC API link.
<code>samtools view -b "\$BAM_URL" chr21 -o {BAM_PATH}</code>	The remote-streaming trick — pulls ONLY chr21's reads from the remote BAM via HTTP range requests, avoiding a 90-150GB full download.
<code>samtools index {BAM_PATH}</code>	Builds the <code>.bai</code> index the pileup-reading code needs for fast positional lookups.
<code>samtools idxstats {BAM_PATH} awk '\$3+\$4 > 0'</code>	Sanity check — lists only chromosomes with actual reads present, confirming we really did only pull chr21.
<code>tabix -p vcf {TRUTH_VCF}</code>	Indexes the truth VCF for fast positional queries (used by <code>pysam.VariantFile</code>).
<code>python3 candidate_generation.py --bam ... --regions_bed ...</code>	Runs Part 3's scan, chr21 → 191,306 filtered candidates.
the region-split cell	Pure Python: sorts candidates by position, slices into 70/15/15% train/val/test with no positional overlap.
<code>python3 real_data.py --candidates ... - -out train.h5</code>	Runs Part 4's tensor extraction for one split, writing the resumable HDF5 cache.

Command	What it does
<code>python3 train_production.py --arch fusion --loss vaf_aware ...</code>	Runs Part 8's training loop.
<code>!cp {CACHE_DIR}/*.h5 /content/cache/</code>	Copies caches to local disk — later found to not be the actual fix (Section 6.4 of the session report), but harmless and still a reasonable practice.
<code>python3 repair_vaf.py --h5 ...</code>	Runs Part 9's in-place VAF fix.

PART 11 — Master FAQ / Criteria Summary

"What decides whether a position even gets considered?" — Candidate generation's three thresholds (Part 3.3): enough reads disagree, in a high enough fraction, at high enough depth, inside a region SEQC2 validated.

"What decides the label (variant or not)?" — Purely whether the position appears in the SEQC2 sSNV truth VCF. Nothing about the model's own predictions or the observed VAF affects the label.

"What decides the VAF value used for weighting/bucketing?" — The real, directly observed alt-allele fraction from the actual reads, computed the same way whether the example is a true positive or a negative.

"Why does the model see two different representations of the same site?" — Because local evidence (exact base match/mismatch at exactly this position) and broader context (is this a structurally unusual region) are genuinely different kinds of information, and no single representation captures both well — that's the entire thesis of the fusion architecture.

"What's actually novel here versus copying DeepVariant or TransSSVs?" — Neither of those fuses a CNN pileup stream with a Transformer context stream via cross-attention for somatic calling; each uses only one representation. The fusion mechanism, plus VAF-aware training, plus the explainability/reporting layer together, is the contribution — not any one individual component in isolation.

"Where are we deliberately running smaller/cheaper than the eventual target?" Pileup window (21 vs. literature's 221), single chromosome (chr21 vs. whole genome), single BAM replicate (vs. multiple available), CNN depth (2 layers, not rigorously tuned). Where we are NOT running smaller than spec: Transformer depth (6 layers, 8 heads — already full target), context window (± 150 bp — already literature-matched).